

QUASE NADA SOFTWARE HOUSE — AULA TÉCNICA

Deploy de um backend Node.js no Oracle Cloud Free Tier

Do `ssh ubuntu@...` ao `/health` respondendo da internet

Projeto: Quase Nada Finanças — agregador financeiro + investimento cripto

Stack: Fastify · Prisma · PostgreSQL · Redis · BullMQ · Docker Compose · Expo

Servidor: Oracle Cloud Infrastructure (OCI) — Ubuntu 22.04 ARM/AMD64 Free Tier

Autor: Anotações compiladas durante o deploy real do server `quase-nada-server-2`

Data: 16 de maio de 2026

Sumário

01	Visão geral: o que estamos deployando e por quê	3
02	Topologia: como as peças conversam	5
03	SSH com chave privada — autenticação sem senha	7
04	Diagnóstico inicial: lendo o estado de uma máquina nova	9
05	Memória virtual (swap): por que 1 GB de RAM não chega	12
06	Instalando Docker do jeito certo	15
07	Firewall em duas camadas: iptables + Security List do OCI	18
08	Transferindo código: <code>tar</code> <code>ssh tar</code> quando não há rsync	22
09	Segredos de produção: <code>openssl rand</code> , heredoc e <code>chmod 600</code>	24
10	Anatomia de um Dockerfile multi-stage	27
11	<code>docker-compose.yml</code> dissecado linha por linha	31
12	Migrations de banco no startup do container	35
13	Os bugs TypeScript que apareceram (e o que aprender com eles)	37
14	Frontend Expo apontando para o backend remoto	41
15	Operação no dia a dia: comandos de manutenção	44
16	O que falta para chamar de "produção de verdade"	46

Apêndices: A. Comandos de referência rápida · B. Glossário de termos

01 Visão geral: o que estamos deployando e por quê

O **Quase Nada Finanças** é um aplicativo mobile (React Native + Expo) cujo backend agrega contas bancárias do usuário via *open finance* (Pluggy), categoriza transações e ainda permite ordens de compra cripto na Binance. Tudo isso roda num servidor Node.js feito com *Fastify*, que conversa com PostgreSQL para persistência e Redis para fila de jobs em background.

O objetivo desta aula é destrinchar o processo de tirar esse backend do seu PC e colocar ele numa VM da Oracle Cloud — uma das poucas nuvens que ainda oferece tier gratuito perpétuo decente — explicando **cada decisão** tomada ao longo do caminho. Não é "siga estes passos"; é "entenda por que esses passos existem".

Por que Oracle Cloud Free Tier?

Existem três motivos práticos:

1. **Always Free:** 2 VMs AMD64 ou até 4 VMs ARM64 Ampere, sem cartão de crédito sendo cobrado depois de 30 dias. AWS/GCP/Azure não dão isso.
2. **IPv4 público fixo:** cada VM ganha um endereço público próprio, o que evita gambiarra de DDNS para deploy de aplicativos mobile.
3. **Banda decente:** 10 TB/mês de saída no plano gratuito. Render/Railway cobram saída a partir do primeiro GB.

A contrapartida: a Oracle não te dá nada de mão beijada. Você recebe uma Ubuntu pelada, sem Docker, sem nginx, com firewall fechado em duas camadas distintas. Tem que saber o que está fazendo.

O que vamos deixar rodando

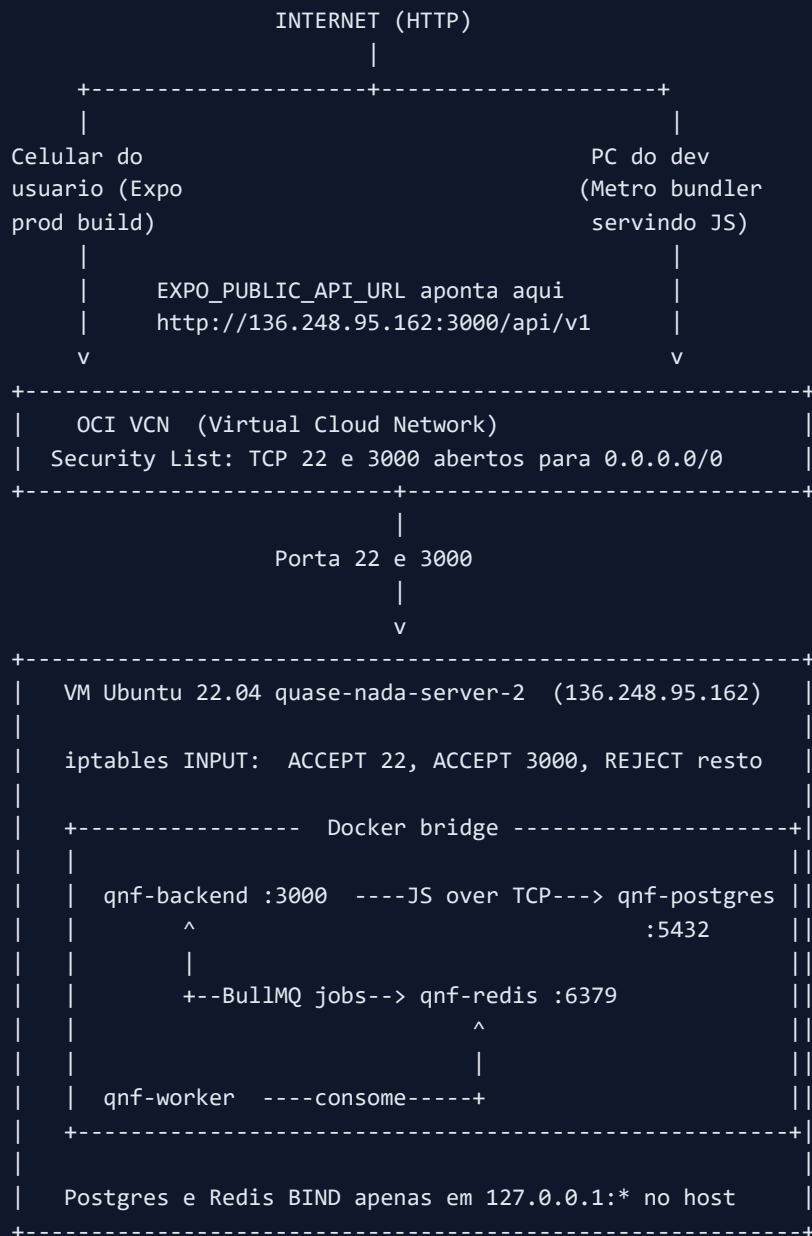
Componente	Imagem Docker	Função	Porta interna
<code>qnf-postgres</code>	<code>postgres:16-alpine</code>	Persistência relacional (usuários, contas, transações)	5432
<code>qnf-redis</code>	<code>redis:7-alpine</code>	Fila BullMQ + cache de cotações Binance	6379
<code>qnf-backend</code>	build local	API HTTP Fastify (auth, contas, transações, dashboards, Binance)	3000
<code>qnf-worker</code>	build local	Processador da fila — sincroniza contas Pluggy a cada 4h	—

Apenas o backend (porta 3000) é exposto para a internet. Postgres e Redis ficam acessíveis somente pela rede privada do Docker.

RECAPITULANDO

Backend em Node + Postgres + Redis + worker, tudo orquestrado por `docker compose`, rodando numa VM Ubuntu gratuita da Oracle, com firewall fechado e API exposta apenas onde necessário.

Antes de mexer em qualquer arquivo, vale entender o caminho que uma requisição faz desde o celular do usuário até o banco e voltar. Esse desenho mental evita 90% dos erros de configuração — você passa a saber *onde* uma falha tem que estar antes de procurar.



Camadas, da fora para dentro

1. **OCI VCN (Virtual Cloud Network)** — é o equivalente da Oracle ao "Security Group" da AWS. Ele bloqueia ou libera tráfego antes mesmo dele chegar na sua VM. Por padrão só 22 (SSH) está aberto. Você tem que abrir 3000 manualmente no console da Oracle (o motivo do único passo manual deste deploy).
2. **iptables da VM** — segundo filtro, agora dentro do sistema operacional. A imagem Ubuntu da Oracle vem com `iptables-persistent` e regras default que *rejeitam* tudo exceto SSH. Adicionamos uma regra `ACCEPT` para 3000.
3. **Bridge do Docker** — uma rede virtual interna criada automaticamente pelo Compose. Os containers se enxergam pelos nomes (`postgres`, `redis`, etc.), o que é magia DNS interna do Docker.
4. **Bind do host** — quando publicamos `"127.0.0.1:5432:5432"` em vez de `"5432:5432"`, o Postgres só responde a conexões originadas *do próprio servidor*. Defesa em camadas: mesmo se alguém burlasse os dois firewalls anteriores, ainda esbarraria nessa terceira barreira.

HEURÍSTICA MENTAL

Quando uma conexão não funciona, faça a pergunta "*qual camada bloqueou?*" e ataque uma de cada vez: `curl 127.0.0.1:PORT` testa o app, `curl 10.0.0.x:PORT` testa a rede interna, `curl IP_PUBLICO:PORT` testa o firewall externo. Quando você sabe qual camada falhou, o conserto é trivial.

SSH com chave privada — autenticação sem senha

Quando você cria uma instância na Oracle Cloud, o console pede para você fazer upload (ou geração) de um par de chaves SSH. A chave **pública** fica registrada no servidor em `/home/ubuntu/.ssh/authorized_keys`; a **privada** fica com você. Essa assimetria é o coração da criptografia de chave pública.

O comando que abre a sessão

```
ssh -i "C:\Users\vinim\.ssh\private_oracle_quase_nada_server2.key" ubuntu@136.248.95.1
```

Parte	O que faz
<code>ssh</code>	Cliente SSH (Secure SHell). No Windows 10/11 vem nativo desde 2019.
<code>-i caminho</code>	<i>Identity file</i> . Diz qual chave privada usar. Sem isso, o cliente tenta as chaves padrão em <code>~/.ssh/</code> (<code>id_rsa</code> , <code>id_ed25519...</code>) e falha.
<code>ubuntu@</code>	Usuário no destino. Em imagens cloud é convenção ter um usuário sem senha mas com sudo NOPASSWD — no Ubuntu da Oracle, é <code>ubuntu</code> .
<code>136.248.95.162</code>	IP público da VM. Poderia ser um hostname (se você configurar DNS), mas IP cru funciona.

Por que não pode usar senha?

Tecnicamente até dá pra habilitar autenticação por senha em SSH. Mas:

- Senhas são vulneráveis a brute force. Em uns 3 minutos depois de subir uma VM com IP público, você já vê tentativas de login automatizadas nos logs (`journalctl -u ssh`). A Oracle desabilita por padrão para evitar isso.
- Chave privada é praticamente impossível de adivinhar — são 256 bits de entropia (ED25519) ou 4096 bits (RSA).

Cuidados com o arquivo `.key`

PERMISSÕES IMPORTANTAM

No Linux/Mac, o cliente SSH **recusa** a chave se o arquivo não estiver com permissão 600 (apenas dono lê e escreve). No Windows o NTFS faz isso por ACL — se aparecer

`Permissions for 'key' are too open`, vá nas propriedades → Segurança → Avançado → Desabilitar herança → manter apenas o seu usuário com Leitura.

O `StrictHostKeyChecking` da primeira conexão

Na primeira vez que você conecta num servidor, o SSH mostra a *fingerprint* (hash da chave pública do servidor) e pergunta se você confia. Ele grava em `~/.ssh/known_hosts`. Da segunda vez em diante, se a fingerprint mudou, ele **recusa conectar** alegando possível ataque MITM. Para automação eu usei `-o StrictHostKeyChecking=no` só na primeira chamada — em produção, jamais.

PEGADINHA

Se você destrói e recria a VM no mesmo IP, a chave do servidor muda e seu SSH vai recusar. Remova a entrada antiga: `ssh-keygen -R 136.248.95.162`. É um erro que assusta gente que nunca viu.

Diagnóstico inicial: lendo o estado de uma máquina nova

Antes de instalar qualquer coisa, eu rodei uma bateria de comandos para entender o que tinha pela frente. Esse hábito separa quem sabe administrar máquinas de quem só copia tutoriais.

O kit de inspeção

```
uname -a           # kernel e arquitetura
lsb_release -a     # distribuição e versão
df -h /            # espaço em disco
free -h           # memória RAM e swap
which docker       # algum docker pré-instalado?
sudo ufw status    # firewall ufw está ativo?
sudo iptables -L INPUT -n --line-numbers # regras de firewall reais
ss -tlnp          # que portas estão escutando?
```

Lendo a saída crítica deste servidor

Comando	Saída relevante	O que isso me disse
<code>uname -a</code>	Linux ... 6.8.0-1049- oracle ... x86_64	Kernel da Oracle (otimizado pra OCI). AMD64, não ARM — confirma a forma da VM. Docker images precisam ser amd64.
<code>lsb_release</code>	Ubuntu 22.04.5 LTS (jammy)	Distro LTS, codinome <code>jammy</code> . Importante porque o repo Docker oficial precisa do codinome para escolher os pacotes certos.
<code>free -h</code>	Mem 956Mi Swap 0B	Sinal de alerta: menos de 1 GB de RAM e zero swap. Postgres+Redis+Node+Worker vão estourar. Tem que adicionar swap antes de qualquer coisa.
<code>df -h /</code>	45G 2.3G used	Sobra disco — 43 GB livres. Imagens Docker são gordas, mas dá folga.
<code>iptables -L INPUT</code>	Regras 1–5 ACCEPT, regra final REJECT	Firewall padrão Oracle. Tudo bloqueado exceto SSH. Vou precisar inserir regra para 3000 antes do REJECT final.
<code>ss -tlnp</code>	Apenas sshd:22 e systemd- resolved:53	Máquina limpa. Nenhum serviço de aplicação rodando.

O que cada um desses comandos significa

`uname -a`

"Unix name, all". Mostra o kernel rodando. O `x86_64` no final é a arquitetura — fundamental para escolher binários compatíveis. Se fosse `aarch64` (ARM), todo o resto seria diferente — algumas imagens Docker não têm build ARM, por exemplo.

`free -h`

Memória em formato humano. As colunas relevantes:

- **total:** RAM física instalada.
- **used:** o que o kernel *tirou* dela.
- **available:** o que efetivamente pode ser alocado por novos processos. Esse é o número que importa.

- **Swap:** memória virtual em disco (próximo módulo).

```
iptables -L INPUT -n --line-numbers
```

Lista as regras da chain *INPUT* (pacotes *entrando*) com números de linha. As regras são avaliadas em ordem; assim que uma casa, o resto não é avaliado. O `-n` mostra IPs e portas numéricos (sem resolver DNS, mais rápido).

CONCEITO: ORDEM EM IPTABLES

Como iptables é "first match wins", inserir uma regra na *posição* certa é essencial. Adicionar com `-A` (append) coloca *no fim* — depois do REJECT final, sem efeito. Use `-I N` (insert na posição N) para colocar antes do REJECT. Foi exatamente o que fizemos.

Memória virtual (swap): por que 1 GB de RAM não chega

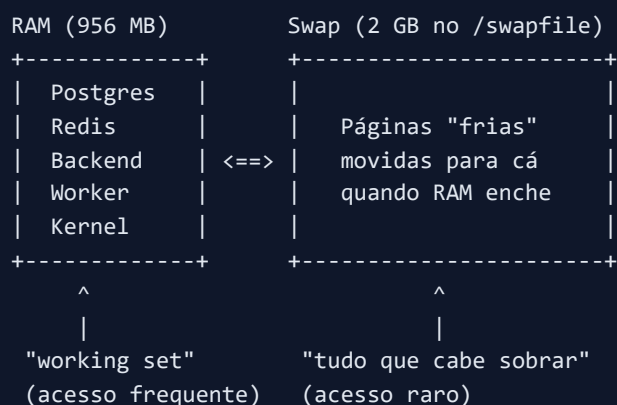
Esta VM tem 956 MB de RAM. Vamos rodar simultaneamente:

- **PostgreSQL 16** — em idle consome ~80 MB; com workload chega fácil a 300 MB.
- **Redis 7** — ~10 MB ocioso, mas com appendonly fsync pode picar.
- **Node.js (Fastify API)** — base do Node V8 são ~50 MB + heap. Em produção comum 150–250 MB.
- **Node.js (worker BullMQ)** — mesma coisa, mais ~150 MB.

Some: ~700 MB no piso, fácil estourar 1 GB em pico. Sem swap, o kernel chama o *OOM killer* (Out Of Memory) e mata o processo mais "guloso" — geralmente o Postgres, justo o que você **não** quer morto.

O que é swap?

Swap é uma extensão da RAM usando o disco. Quando a RAM enche, o kernel move páginas pouco usadas pra um arquivo (ou partição) chamado *swap*, liberando RAM real. É **muito** mais lento que RAM (SSDs modernos são ~100× mais lentos que DDR4), mas evita o OOM killer.



Os passos que executei

```

sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
sudo sysctl vm.swappiness=10

```

Comando	Por que existe
<code>fallocate -l 2G /swapfile</code>	Aloca 2 GB contíguos. É <i>much</i> mais rápido que <code>dd if=/dev/zero</code> antigamente usado — em vez de escrever zeros, ele só reserva no filesystem.
<code>chmod 600 /swapfile</code>	Só o root pode ler. Crítico: dados sensíveis em memória (chaves, senhas) podem acabar no swap; outros usuários não podem espiar.
<code>mkswap /swapfile</code>	"Formata" o arquivo como espaço de swap (escreve a estrutura interna que o kernel espera).
<code>swapon /swapfile</code>	Ativa o swap <i>agora</i> . Sem persistência — sumiria no reboot.
<code>echo '... ' >> /etc/fstab</code>	Persiste. <code>fstab</code> (filesystem table) é lido no boot e ativa swap/partições automaticamente.
<code>sysctl vm.swappiness=10</code>	Diz pro kernel: "use swap só quando for muito necessário". Default no Ubuntu é 60 (agressivo). 10 é conservador — bom para servidores.

REGRA DE BOLSO

Swap igual à RAM é o velho conselho de quando RAM era cara. Hoje, **swap = 2× RAM** para máquinas pequenas (<2 GB) é mais útil. Para máquinas grandes (>16 GB), 4–8 GB de swap basta.

Validando o resultado

```

free -h
# Mem: 956Mi total ... 599Mi available
# Swap: 2.0Gi total · 0B used · 2.0Gi free

```

Swap criado e ainda vazio (como esperado em idle). Quando o workload subir, o kernel começa a usar.

SWAP NÃO É SOLUÇÃO, É AIRBAG

Se sua aplicação *regularmente* usa swap, ela vai estar lenta demais. Swap é colchão para picos eventuais; uso constante = upgrade de RAM (ou shape maior na Oracle).

Existem três formas de instalar Docker no Ubuntu:

Método	Vantagem	Problema
<code>apt install docker.io</code>	Um comando só	Vem desatualizado (versão do repo Ubuntu, atrasa meses). Sem o plugin oficial <code>docker compose</code> .
<code>curl get.docker.com sh</code>	Funciona em qualquer distro	Script grande, opaco, baixa e executa código root. Recomendado apenas para experimentação.
Repo oficial da Docker Inc.	Sempre atualizado, plugin Compose incluso, processo auditável	Mais comandos pra digitar

Fui pelo terceiro caminho. Vale entender o que cada bloco faz.

1) Pré-requisitos e chaves GPG

```
sudo apt-get update -qq
sudo apt-get install -y ca-certificates curl gnupg lsb-release rsync iptables-persistent
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg --yes
```

O `apt` usa *assinaturas GPG* para verificar que os pacotes vieram realmente da fonte declarada — sem isso, qualquer um que controle o caminho de rede poderia injetar pacotes maliciosos. A chave pública da Docker é baixada e convertida em formato binário (`--dearmor`) para uma pasta confiável.

2) Adicionar o repositório

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list
```

Essa linha gigante diz para o `apt` :

- `deb` : tipo de repositório binário.
- `arch=$(dpkg --print-architecture)` : pega `amd64` ou `arm64` dinamicamente. Importante para a portabilidade do script.
- `signed-by=...` : só aceita pacotes assinados por aquela chave que baixamos.
- `$(lsb_release -cs)` : substitui pelo codinome (`jammy` nesta VM). Sem isso o apt baixaria pacotes do Debian errado.

3) Instalar os pacotes

```
sudo apt-get update -qq
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin
```

Pacote	O que é
<code>docker-ce</code>	O daemon — processo de longa duração que gerencia os containers.
<code>docker-ce-cli</code>	O cliente (<code>docker ps</code> , <code>docker build</code> ...). Conversa com o daemon via socket Unix.
<code>containerd.io</code>	Runtime de containers de baixo nível. O daemon Docker delega para ele.
<code>docker-buildx-plugin</code>	BuildKit moderno, build multi-arquitetura, cache eficiente.
<code>docker-compose-plugin</code>	Permite usar <code>docker compose</code> (sem hífen) — versão moderna, escrita em Go, integrada ao CLI.

DOCKER-COMPOSE VS DOCKER COMPOSE

`docker-compose` (com hífen) é a v1, escrita em Python, descontinuada. `docker compose` (sem hífen) é a v2, plugin Go nativo. APIs idênticas em 95% dos casos. Use **sem hífen** sempre que possível.

4) Permissões e habilitação


```
sudo usermod -aG docker ubuntu
sudo systemctl enable --now docker
```

O `usermod -aG docker ubuntu` adiciona o usuário `ubuntu` ao grupo `docker`, que tem permissão para falar com o socket do daemon — assim você não precisa de `sudo` antes de cada `docker`.

PEGADINHA QUE PEGA MUITA GENTE

A mudança de grupo só vale para *novas sessões*. Na mesma sessão SSH em que você executou o `usermod`, o `docker ps` sem `sudo` ainda dá *permission denied*. Saia e entre de novo (`exit` + reconectar), ou use `newgrp docker` pra forçar reload. Por isso ainda usei `sudo docker compose` nos comandos do mesmo SSH — era a sessão original.

O `systemctl enable --now docker` faz duas coisas: *enable* liga o daemon no boot, *--now* também inicia agora.

07 Firewall em duas camadas: iptables + Security List do OCI

Este é o ponto que mais confunde quem nunca usou OCI. Existem **dois** firewalls separados em série, e os dois precisam liberar o tráfego.



Camada 1: iptables (faço pelo SSH)

```
sudo iptables -I INPUT 5 -p tcp -m state --state NEW -m tcp --dport 3000 -j ACCEPT
sudo netfilter-persistent save
```

Esmiuçando:

- `-I INPUT 5` — *Insert* na chain INPUT, na **posição 5**. A posição 6 era o REJECT final. Se eu tivesse usado `-A` (append) entraria na posição 7, depois do REJECT — inútil.
- `-p tcp` — pacotes TCP.
- `-m state --state NEW` — match no módulo *state*, somente pacotes que iniciam conexão. Pacotes de conexões já estabelecidas são tratados pela regra 1, mais rápida.

- `--dport 3000` — destino: porta 3000.
- `-j ACCEPT` — ação: aceitar.

O `netfilter-persistent save` grava o conjunto atual em `/etc/iptables/rules.v4` para que a regra sobreviva ao reboot. Por isso instalamos `iptables-persistent` lá atrás.

Camada 2: VCN Security List (manual no console OCI)

Esta camada não dá pra alterar via SSH. É um recurso de rede da Oracle, configurável só pelo Console OCI ou pela CLI da Oracle (que requer mais setup).

Caminho no console:

1. **Networking** → **Virtual Cloud Networks**
2. Abrir o VCN da sua instância
3. Aba **Security Lists** → default
4. **Add Ingress Rule:**

Campo	Valor
Source Type	CIDR
Source CIDR	<code>0.0.0.0/0</code>
IP Protocol	TCP
Source Port Range	(vazio = qualquer)
Destination Port Range	<code>3000</code>
Description	QNF backend (Fastify)

Por que duas camadas?

Defesa em profundidade. Se uma falha (config errada, bug, comprometimento), a outra ainda protege.

Cenário	iptables só	VCN só	Os dois
Admin esquece de iptables	—	Bloqueia	Bloqueia
Vulnerabilidade no kernel	Vazaria	Bloqueia	Bloqueia
Erro no Security List	Bloqueia	Vazaria	Bloqueia

VALIDAÇÃO RÁPIDA DAS DUAS CAMADAS

De **dentro** da VM: `curl 127.0.0.1:3000/health`. Se responde → app ok.

De **fora** da VM: `curl http://IP_PUBLICO:3000/health`. Se timeouta → uma das duas camadas bloqueando.

Para saber qual: `sudo iptables -L INPUT -n | grep 3000`. Se mostra a regra, então é o VCN.

Por que não usar UFW?

UFW é só um "wrapper amigável" em cima de iptables. Em servidores cloud com regras herdadas (como a Oracle, que já vem configurada), misturar UFW pode dar conflito — ele reescreve as chains. Para evitar surpresa, deixei UFW como veio (`inactive`) e mexi direto em iptables.

08 Transferindo código: `tar` | `ssh tar` quando não há `rsync`

Para enviar o código do meu PC para a VM, existem várias opções:

1. `git clone` direto no servidor — mas exige que o repo esteja no GitHub e que o servidor tenha credencial pra puxar.
2. `scp -r pasta` — funciona, mas reenvia tudo a cada execução, sem deltas.
3. `rsync -avz` — manda só o que mudou, eficiente. Mas no Windows não vem instalado.
4. `tar -czf - . | ssh remote "tar -xzf -"` — usa só `tar` e `ssh`, ambos nativos. Foi o que usei.

O comando exato

```
tar --exclude=node_modules --exclude=dist --exclude=.env --exclude=.env.local \
  -czf - . \
| ssh -i "... key" ubuntu@136.248.95.162 \
  "tar -xzf - -C /home/ubuntu/quase-nada-financas/backend"
```

Como isso funciona

1. `tar -czf - .` — empacota o diretório atual (`.`), comprime com gzip (`-z`) e escreve em **stdout** (`-f -`). Nada toca o disco.
2. `|` — o pipe envia o stdout do `tar` para o stdin do próximo comando.
3. `ssh ... "tar -xzf - -C destino"` — abre conexão SSH, executa do outro lado o `tar` reverso: lê de stdin (`-f -`), descompacta (`-x`) dentro do destino (`-C`).

Resultado: arquivo nenhum criado em lugar nenhum. O conteúdo flui pela rede já comprimido, é descomprimido direto no destino. Elegante.

Os `--exclude`

Padrão	Motivo de excluir
<code>node_modules</code>	Centenas de MB. Vai ser regenerado pelo <code>npm ci</code> dentro do Dockerfile, com versões corretas pra Alpine Linux.
<code>dist</code>	Build local pode estar desatualizado ou compilado pra Windows. O Dockerfile faz <code>npm run build</code> dentro do container.
<code>.env</code> e <code>.env.local</code>	Segredos! Não devem sair do seu PC. O <code>.env</code> de produção é gerado <i>no servidor</i> .

CLOCK SKEW NOS LOGS DO TAR

Apareceu `tar: tsconfig.json: time stamp is 76s in the future` em alguns runs. Isso é porque o relógio do meu PC (Windows) e da VM (Linux) estão dessincronizados em poucos segundos. É um *warning* inofensivo do tar — ele alerta que mtime do arquivo está no "futuro" do ponto de vista da máquina destino. Funciona normalmente. Em ambientes estritos, sincronize com NTP: `sudo timedatectl set-ntp true`.

`chmod 600`

O backend precisa de várias variáveis de ambiente: secrets do JWT, chave de criptografia para tokens da Pluggy, credenciais de banco, URLs de APIs externas. Em desenvolvimento usamos valores placeholder; em produção tudo isso precisa ser **aleatório, único e privado**.

Geração de segredos com `openssl`

```
openssl rand -hex 32
```

Saída: 64 caracteres hexadecimais. Por trás dos panos:

- `rand 32` — gera 32 bytes (256 bits) de entropia criptográfica usando `/dev/urandom` (Linux) ou `CryptGenRandom` (Windows).
- `-hex` — codifica em hexadecimal. Cada byte vira 2 chars → 64 chars.

POR QUE 256 BITS?

É o padrão da indústria para chaves simétricas. 128 bits ainda é considerado seguro contra força bruta clássica, mas 256 bits é o que JWT HS256, AES-256, ChaCha20 esperam — e protege contra ataques futuros com computação quântica (que enfraquecem chaves simétricas pela metade, mas 256 → 128 ainda é seguro).

Variáveis geradas

Variável	Uso	Tamanho
<code>JWT_ACCESS_SECRET</code>	Assina tokens de acesso (15min)	32 bytes
<code>JWT_REFRESH_SECRET</code>	Assina refresh tokens (30 dias)	32 bytes
<code>ENCRYPTION_KEY</code>	Criptografa chaves Pluggy/Binance no banco (AES-GCM)	32 bytes

ACCESS & REFRESH: POR QUE DOIS SECRETS?

Separar permite invalidar todos os refresh tokens (rotação de chave) sem deslogar usuários ativos. Também limita o "raio de explosão" se um deles vazar.

Heredoc para gerar o arquivo

```
JWT_A=$(openssl rand -hex 32)
JWT_R=$(openssl rand -hex 32)
ENC=$(openssl rand -hex 32)

cat > .env <<EOF
JWT_ACCESS_SECRET=${JWT_A}
JWT_REFRESH_SECRET=${JWT_R}
ENCRYPTION_KEY=${ENC}
NODE_ENV=production
...
EOF
```

O `<<EOF ... EOF` é um **here-document**: o shell trata tudo entre os `EOF` como input do comando anterior. Como não usei `'EOF'` entre aspas, as expansões `${JWT_A}` são interpretadas — exatamente o que queremos.

ASPAS NO EOF IMPORTAM

`<<EOF` = expande variáveis. `<<'EOF'` = literal, não expande. Se você quer gravar literalmente uma string `$VAR` num arquivo, use aspas simples.

Permissões do arquivo

```
chmod 600 .env
```

Octal 600 = leitura+escrita só pro dono. Outros usuários nem conseguem ler. Crucial pra arquivos com segredos.

RECAPITULANDO

Segredos gerados **localmente no servidor** com entropia criptográfica, gravados em arquivo com permissão restrita, lidos pelo container via `env_file`: no compose. Nunca trafegam pela rede, nunca vão pro git.

Anatomia de um Dockerfile multi-stage

Dockerfile é uma receita: cada instrução cria uma camada. *Multi-stage* significa ter vários `FROM` num mesmo arquivo — cada um inicia um "estágio" independente, e você pode copiar arquivos entre eles.

Por que isso importa? Porque a imagem final tem que ser pequena. Não queremos enviar o npm cache, devDependencies ou source code TypeScript para produção — só o que *roda*.

O Dockerfile completo

```
# Stage 1: deps
FROM node:20-alpine AS deps
WORKDIR /app
RUN apk add --no-cache libc6-compat openssl
COPY package.json package-lock.json* ./
RUN npm ci

# Stage 2: build
FROM node:20-alpine AS build
WORKDIR /app
RUN apk add --no-cache openssl
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN npx prisma generate
RUN npm run build

# Stage 3: runtime
FROM node:20-alpine AS runtime
WORKDIR /app
RUN apk add --no-cache openssl tini
ENV NODE_ENV=production
COPY package.json package-lock.json* ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=build /app/dist ./dist
COPY --from=build /app/prisma ./prisma
COPY --from=build /app/node_modules/.prisma ./node_modules/.prisma
COPY --from=build /app/node_modules/@prisma ./node_modules/@prisma
COPY --from=build /app/node_modules/prisma ./node_modules/prisma
EXPOSE 3000
ENTRYPOINT ["/sbin/tini", "--"]
CMD ["node", "dist/server.js"]
```

Estágio 1: `deps` — instala TODAS as dependências

O estágio `deps` existe pra que o Docker cache as `node_modules` separadamente. Como só copiei `package.json` e `package-lock.json`, se o código mudar mas as `deps` não, esse estágio fica cacheado e build é instantâneo.

O `apk add libc6-compat openssl` instala libs nativas que algumas dependências Node precisam — Prisma usa OpenSSL para gerar engines.

Estágio 2: `build` — compila TypeScript → JavaScript

Aqui copio o código todo (`COPY . .`) e:

- `npx prisma generate` — gera o cliente Prisma a partir do `schema.prisma`. Esse cliente é code-generated, então tem que rodar a cada build.
- `npm run build` — invoca `tsc`, produzindo `dist/` com JS plain.

Estágio 3: `runtime` — imagem final, magra

Note: este estágio começa *do zero* (`FROM node:20-alpine`), **não** herda nada dos anteriores. Por quê?

- Não queremos os arquivos `.ts`, `tsconfig.json`, `devDependencies`.
- Faço `npm ci --omit=dev` só com prod deps. Aí copio do estágio `build` apenas o que preciso:

O que copio do build	Por quê
<code>dist/</code>	O código JS compilado, o que roda em produção.
<code>prisma/</code>	Schemas e migrations. <code>prisma migrate deploy</code> precisa deles.
<code>node_modules/.prisma</code>	Engines compiladas pro alvo Alpine.
<code>node_modules/@prisma</code>	Pacote do cliente Prisma.
<code>node_modules/prisma</code>	O CLI do Prisma. Esta linha eu adicionei depois — explico no próximo módulo.

`tiny` como entryptoint

`tiny` é um *init* minimalista. Sem ele, o Node roda como PID 1 dentro do container e não responde direito a SIGTERM (não reapa filhos zumbis). Com `tiny`, sinais funcionam, processos zumbis são limpos, shutdown é gracioso.

Por que Alpine?

- Imagem base de ~5 MB (vs ~80 MB do Debian slim, ~250 MB do Debian completo).
- Build mais rápido, push pra registry mais rápido, pull em produção mais rápido.
- Usa `musl libc` em vez de `glibc` — daí o `libc6-compat` no estágio 1 (pra deps que esperam `glibc`).

O BUG QUE APARECEU NO DEPLOY

Eu copiava só `.prisma` e `@prisma`, mas **não** o `prisma` (o CLI). Como o `command` no `compose` roda `npx prisma migrate deploy` no startup, e o CLI tava ausente, o `npx` tentaria baixar — em rede ruim, falha. **Solução:** linha adicional `COPY --from=build /app/node_modules/prisma ./node_modules/prisma`. Detalhe sutil que não aparece em tutorial nenhum até você bater.

11 docker-compose.yml dissecado linha por linha

Compose é declarativo: você descreve o estado desejado e o Docker se vira pra alcançar. Nosso arquivo final, com comentários:

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: qnf-postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: qnf_user
      POSTGRES_PASSWORD: qnf_pass
      POSTGRES_DB: qnf_db
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - qnf_pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U qnf_user -d qnf_db"]
      interval: 5s
      timeout: 5s
      retries: 10
```

Atributos importantes

Atributo	Função
<code>image</code>	Imagem oficial do Postgres na variante Alpine, versão 16.
<code>container_name</code>	Nome fixo (em vez do default <code>projeto_servico_1</code>). Útil pra logs e scripts.
<code>restart: unless-stopped</code>	Se o container cair, sobe sozinho. Não sobe se você parou manualmente. Mata 90% dos "deu falha de madrugada".
<code>environment</code>	Variáveis lidas pelo entrypoint do Postgres pra criar usuário/banco no primeiro start.
<code>ports: 127.0.0.1:5432:5432</code>	Bind apenas em loopback . Nem o iptables vai ver tráfego externo pra 5432.
<code>volumes:</code> <code>qnf_pgdata:/var/lib/postgresql/data</code>	Persistência! Sem isso, dados somem ao recriar o container. <code>qnf_pgdata</code> é um named volume gerenciado pelo Docker.
<code>healthcheck</code>	Testa periodicamente se o serviço está pronto. <code>pg_isready</code> retorna 0 quando o Postgres aceita conexões.

O serviço backend

```
backend:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: qnf-backend
  restart: unless-stopped
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  env_file:
    - .env
  environment:
    DATABASE_URL: postgresql://qnf_user:qnf_pass@postgres:5432/qnf_db?schema=public
    REDIS_URL: redis://redis:6379
  ports:
    - "3000:3000"
  command: ["sh", "-c", "npx prisma migrate deploy && node dist/server.js"]
```

Pontos sutis

`depends_on` **com** `condition`

A versão antiga do compose só esperava o *start* dos serviços, não a *saúde*. A nova permite "service_healthy" — o backend só sobe depois que postgres e redis passaram no healthcheck. Sem isso, o backend tentaria abrir conexão antes do Postgres aceitar e crashava.

`env_file` **VS** `environment`

Os dois coexistem. As variáveis em `environment` têm **prioridade** e sobrescrevem o `.env`. Por isso o `DATABASE_URL` no compose usa o hostname `postgres` (resolvido pela rede Docker) em vez de `localhost` — mesmo que o `.env` tenha outro valor pra dev local, em produção o compose vence.

`command` **sobrescrevendo o** `CMD`

O Dockerfile termina com `CMD ["node", "dist/server.js"]`. O compose substitui isso por `sh -c "npx prisma migrate deploy && node dist/server.js"` — antes de subir o servidor, aplica migrations. Padrão muito comum.

Volumes nomeados

```
volumes:  
  qnf_pgdata:  
  qnf_redisdata:
```

Declarar aqui no nível raiz *cria* os volumes. O Docker gerencia onde eles vivem no host (`/var/lib/docker/volumes/...`). A vantagem sobre bind mounts (`./data:/var/lib/...`) é portabilidade — funciona igual em qualquer host, sem se preocupar com permissões de pasta.

BACKUP DESSES VOLUMES

Esse é o conteúdo crítico do sistema. Pra fazer backup:

```
docker run --rm -v qnf_pgdata:/data -v $(pwd):/backup alpine tar czf  
/backup/pgdata.tgz /data
```

Roda um container temporário, monta o volume e o pwd atual, e empacota tudo.

Migrations de banco no startup do container

O Prisma usa um workflow declarativo: você modifica o `schema.prisma`, gera uma migration com `prisma migrate dev` em desenvolvimento (cria arquivo SQL versionado), commita o arquivo, e em produção **aplica** a migration com `prisma migrate deploy`.

Por que rodar no startup?

Existem duas escolas:

1. **Migration como job separado** antes do deploy. Mais seguro: se a migration falhar, o app antigo continua rodando.
2. **Migration no startup do container**. Mais simples: deploy é "atomic" — sobe junto. Adequado para projetos pequenos como o nosso.

Adotei a 2ª: `command: ["sh", "-c", "npx prisma migrate deploy && node dist/server.js"]`.

O que aconteceu na primeira execução

```
Prisma schema loaded from prisma/schema.prisma
Datasource "db": PostgreSQL database "qnf_db" ...
1 migration found in prisma/migrations
Applying migration `20260512161947_init`
The following migration(s) have been applied:
migrations/
├─ 20260512161947_init/
│   └─ migration.sql
All migrations have been successfully applied.
Server listening at http://0.0.0.0:3000
```

Migration aplicada, depois o servidor sobe. Perfeito.

A race condition com o worker

O worker também conecta no Postgres. Como ele e o backend sobem em paralelo (ambos esperam só os healthchecks de DB e Redis, não *um do outro*), o worker tentou ler a tabela `ConnectedAccount` antes do backend rodar a migration. Resultado nos logs:


```
code: 'P2021',
clientVersion: '5.22.0',
meta: { modelName: 'ConnectedAccount', table: 'public.ConnectedAccount' }
Node.js v20.20.2
```

P2021 = "the table does not exist in the current database". O processo Node saiu.

O salvador foi o `restart: unless-stopped`: o container caiu, o Docker o reanimou automaticamente segundos depois. Aí a migration já havia rodado, a tabela existia, e o worker rodou normal:

```
Pluggy sync worker ready
Enqueued Pluggy sync jobs, count: 0
```

A MANEIRA "ELEGANTE" DE RESOLVER

Em vez de depender do restart, dá pra ter um serviço dedicado de migration que sobe primeiro, ou um `command` no worker que aguarde a tabela existir (com retry loop). Pra um projeto pequeno, restart policy é trade-off aceitável.

Os bugs TypeScript que apareceram (e o que aprender com eles)

Ao tentar o primeiro `docker compose build`, o estágio `build` falhou com 4 erros distintos do TypeScript. Vale entender cada um — são padrões que aparecem em qualquer projeto sério.

Erro 1: `rootDir` versus `include`

```
error TS6059: File '/app/prisma/seed.ts' is not under 'rootDir' '/app/src'.
```

O `tsconfig.json` tinha:

```
{
  "compilerOptions": {
    "rootDir": "src",
    "outDir": "dist"
  },
  "include": ["src/**/*", "prisma/**/*"]
}
```

Por que dá erro

`rootDir` diz onde fica a raiz dos arquivos fonte; o TS usa pra calcular a estrutura de pastas em `outDir`. Se você incluir um arquivo *fora* do `rootDir`, ele não sabe pra onde gerar — daria conflito de paths.

A correção

Removi `prisma/**/*` do `include`. O arquivo `prisma/seed.ts` é executado direto via `tsx` `prisma/seed.ts` (em dev) ou `prisma db seed` — não precisa ser compilado para JS.

HEURÍSTICA

Se um arquivo TS só roda via `tsx` ou `ts-node`, não inclua no build. Se vai compilar pra JS estático, inclua e mantenha dentro do `rootDir`.

Erro 2: Generic explícito do Fastify quebrando o `typeProvider`

```
error TS2322: Type 'FastifyInstance<Server, ... , Logger>'
is not assignable to type 'FastifyInstance<... , FastifyBaseLogger>'.
```

O código fazia:

```
import Fastify, { FastifyInstance } from "fastify";
export async function buildApp(): Promise<FastifyInstance> { ... }
```

Por que dá erro

Quando você passa `logger: pinoInstance` ao Fastify, ele *infere* um tipo mais específico (`Logger<...>` do pino) em vez do genérico `FastifyBaseLogger`. Aí o tipo retornado pelo Fastify não casa exatamente com `FastifyInstance` genérico declarado no return type — os generics são *invariant* em TypeScript.

A correção

Removi a anotação explícita de return type. TypeScript infere o tipo correto sozinho.

```
import Fastify from "fastify";
export async function buildApp() { ... }
```

PRINCÍPIO

Não anote tipos que o TS pode inferir. Anotações explícitas são úteis em APIs públicas (assinaturas de funções exportadas que outros consomem) mas viram dor de cabeça quando libs evoluem os tipos.

Erro 3: `Prisma.InputJsonValue` em vez de `Record`

```
error TS2322: Type 'Record<string, unknown>' is not assignable to type
'NullableJsonNullValueInput | InputJsonValue | undefined'.
```

Tínhamos em dois lugares:

```
responseData: response as unknown as Record<string, unknown>,
```

Por que dá erro

Prisma tem um tipo específico (`Prisma.InputJsonValue`) que aceita strings, números, booleans, null, arrays e objetos JSON-serializáveis — recursivo. `Record<string, unknown>` é mais permissivo (aceita Date, função, símbolo...) e portanto não satisfaz o tipo Prisma.

A correção

```
import { Prisma } from "@prisma/client";
...
responseData: response as unknown as Prisma.InputJsonValue,
```

GENERATORS E TIPOS

O Prisma **gera** seus tipos a partir do `schema.prisma`. Quando o schema muda, rode `npx prisma generate` pra atualizar. Erros de tipo "absurdos" depois de mexer no schema geralmente são generator desatualizado.

Erro 4: `noUnusedParameters` e o `_`

```
error TS6133: 'userId' is declared but its value is never read.
```

O método `resolveCategoryId(userId, ptx, ...)` recebia `userId` mas não usava. O `tsconfig` tem `"noUnusedParameters": true`, que reclama de parâmetros não-utilizados.

A correção

Renomeei pra `_userId`. O TS, por convenção, ignora parâmetros começando com underscore — sinaliza que o autor sabe que não usa, mas o parâmetro precisa estar ali (talvez por implementar uma interface, ou compatibilidade futura).

```
private async resolveCategoryId(
  _userId: string,    // foi userId
  ptx: PluggableTransaction,
  ...
) { ... }
```

OS 4 ERROS, SINTETIZADOS

1. Não inclua no build TS arquivos fora do rootDir.
2. Não anote tipos quando inferência funciona melhor.
3. Use os tipos específicos da lib (Prisma, Fastify, etc.) em vez de `Record / any`.
4. Use `_` pra silenciar parâmetros conscientemente ignorados.

Frontend Expo apontando para o backend remoto

O app é React Native + Expo. Em desenvolvimento, o app no celular se conecta à API através de uma URL definida em variável de ambiente. Antes do nosso deploy, essa URL era `http://192.168.x.x:3000` (IP local do PC). Agora vira `http://136.248.95.162:3000/api/v1` — o IP público da VM Oracle.

Como Expo lê variáveis em build time

O Expo carrega variáveis via `app.config.js`:

```
function loadLocalEnvFile() {
  const envPath = path.join(__dirname, '.env.local');
  if (!fs.existsSync(envPath)) return;
  const lines = fs.readFileSync(envPath, 'utf8').split(/\r?\n/);
  for (const line of lines) {
    // parse KEY=VALUE
    process.env[key] = value;
  }
}
loadLocalEnvFile();

module.exports = {
  expo: {
    extra: {
      apiUrl: process.env.EXPO_PUBLIC_API_URL,
      appVariant: variant
    }
  }
};
```

O `.env.local` que escrevi

```
APP_VARIANT=development
EXPO_PUBLIC_API_URL=http://136.248.95.162:3000/api/v1
```

Variável	Função
APP_VARIANT=development	Faz o <code>app.config.js</code> usar o bundle ID <code>app.quasenada.financas.dev</code> e ativar <code>NSAllowsArbitraryLoads</code> (permite HTTP no iOS).
EXPO_PUBLIC_API_URL	URL absoluta da API. O prefixo <code>EXPO_PUBLIC_</code> é convenção do Expo — vars com esse prefixo são incluídas no bundle JS final (em vez de só estarem em build time).

Por que `NSAllowsArbitraryLoads` é necessário

Desde iOS 9, o App Transport Security (ATS) bloqueia conexões HTTP (não-HTTPS) por padrão. Pra dev contra um backend sem certificado, precisamos permitir. No `app.config.js`:

```
infoPlist: {
  ... (isDevelopment ? {
    NSAppTransportSecurity: {
      NSAllowsArbitraryLoads: true,
      NSAllowsLocalNetworking: true
    }
  } : {})
}
```

Só aplicado quando `APP_VARIANT=development`. Em produção (preview/release), HTTPS é obrigatório.

EM PRODUÇÃO ISSO VIRA HTTPS

Antes de empacotar uma versão real pra App Store, precisa colocar Caddy/Nginx + Let's Encrypt na frente do backend, e mudar a URL pra `https://api.quasenada.app/api/v1`. Sem isso, ATS bloqueia tudo silenciosamente no iOS.

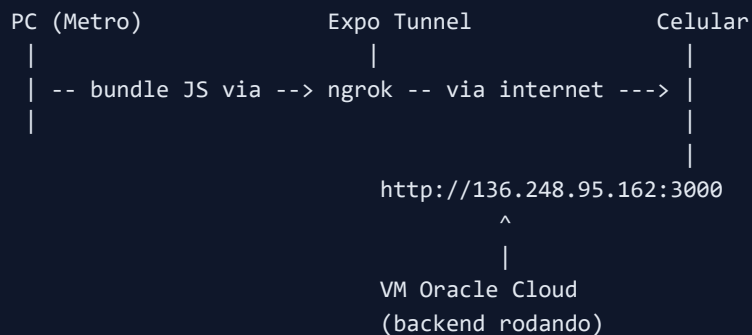
Iniciando o Metro

```
cd projetos\quase-nada-financas\frontend
npm install
npm run start:dev-client:tunnel
```

O que cada flag significa

Flag	Para que serve
<code>--dev-client</code>	Conecta a um development build do app (gerado via EAS). Necessário porque usamos libs nativas (expo-secure-store, expo-local-authentication, pluggify-connect) que não rodam no Expo Go.
<code>--tunnel</code>	Cria um túnel ngrok-style: o Metro vira acessível de qualquer lugar, não só na sua rede local. Útil quando o backend tá em outro lugar (como agora).

Topologia de dev neste cenário



Repare: o tunnel é só pra **servir o JS bundle**. O JS executando no celular fala **direto** com o backend Oracle. Não há proxy no meio das chamadas de API — o que torna o desenvolvimento muito próximo de produção.

Operação no dia a dia: comandos de manutenção

Cheat sheet pra ter ao lado.

Conectar e navegar

```
ssh -i "C:\Users\vinim\.ssh\private_oracle_quase_nada_server2.key" ubuntu@136.248.95.1  
cd /home/ubuntu/quase-nada-financas/backend
```

Status dos containers

```
sudo docker compose ps  
sudo docker stats --no-stream      # CPU, RAM, I/O em tempo real
```

Logs

```
sudo docker compose logs -f backend      # segue logs do API  
sudo docker compose logs -f worker      # segue logs do worker  
sudo docker compose logs --tail=200 backend
```

Mudou o `.env` (ex: pôs Pluggy real)

```
nano .env                                # edita  
sudo docker compose restart backend worker  
curl 127.0.0.1:3000/health
```

Atualizar código (sincronizar do PC)

```
# No PC, na pasta backend:
tar --exclude=node_modules --exclude=dist -czf - . \
| ssh -i "... key" ubuntu@136.248.95.162 \
  "tar -xzf - -C /home/ubuntu/quase-nada-financas/backend"

# Na VM:
sudo docker compose build
sudo docker compose up -d
```

Rebuild completo (reset hard)

```
sudo docker compose down
sudo docker compose up -d --build --remove-orphans
```

Investigar o banco diretamente

```
sudo docker exec -it qnf-postgres psql -U qnf_user -d qnf_db
# Já dentro do psql:
\dt                # lista tabelas
\d "User"           # descreve a tabela User
SELECT COUNT(*) FROM "User";
\q                 # sai
```

Limpar imagens órfãs (libera disco)

```
sudo docker system prune -af --volumes
```

CUIDADO COM `--VOLUMES`

Sem `--volumes` o prune é seguro (só remove imagens e containers parados). Com `--volumes` ele apaga volumes não referenciados — pode comer dados. Só rode `--volumes` se souber exatamente o que tá fazendo.

Backup do Postgres

dump SQL

```
sudo docker exec qnf-postgres pg_dump -U qnf_user qnf_db > backup_$(date +%F).sql
```

dump binário (mais eficiente, melhor pra restore parcial)

```
sudo docker exec qnf-postgres pg_dump -U qnf_user -F c qnf_db > backup_$(date +%F).dum
```

O que falta para chamar de "produção de verdade"

O que temos é "produção funcional" mas ainda não "produção robusta". Lista priorizada:

1. HTTPS (urgente antes de submeter pra App Store)

Botar um reverse proxy (Caddy ou Nginx) na frente do backend, com certificado Let's Encrypt automático.

Caddy é o caminho mais curto — ele faz HTTPS automático com 3 linhas de config:

```
api.quasenada.app {  
  reverse_proxy localhost:3000  
}
```

Requer:

- Domínio próprio com registro A apontando pro IP da VM.
- Abrir 80 e 443 (iptables + Security List).
- Fechar 3000 externamente — só o Caddy precisa enxergar.

2. Backup automatizado do volume Postgres

Cron diário fazendo `pg_dump` e enviando pra Object Storage da Oracle (gratuito até 10 GB).

3. Monitoramento de uptime

Uptime Kuma em outro container, pingando `/health` a cada 60s. Envia alerta no Telegram/Email se cair.

4. Logs centralizados

Hoje os logs ficam dentro de cada container. Pra debug de problemas pretéritos, vale agregar — Loki + Grafana (ou só `logrotate` com retenção).

5. Pipeline CI/CD

Push para `main` dispara: `npm test` → `docker build` → push pra GitHub Container Registry → SSH na VM → `docker pull` + `up -d`. O `DEPLOY.md` do projeto já esboça isso.

6. Hardening do SSH

- Mudar porta padrão (22 → 4022 por exemplo) — reduz ruído de bots.
- Instalar `fail2ban` — banimento automático de IPs que tentam força bruta.
- Manter `PasswordAuthentication no` no `/etc/ssh/sshd_config`.

7. Rate limiting decente no Fastify

O `.env` tem `LOGIN_RATE_LIMIT_MAX` e `ORDER_RATE_LIMIT_MAX` — verifique se estão sendo aplicados no código. Plugin `@fastify/rate-limit` resolve elegantemente.

8. Substituir credenciais Pluggy placeholder

Hoje estão como `replace-with-real-pluggy-client-id`. Quando obtidas no dashboard da Pluggy, edite o `.env` e restart os containers.

MANTRA

Cada item da lista de "produção de verdade" é fácil isoladamente, mas precisa fazer um por um. Resista à vontade de "implementar tudo agora" — vai cansar e nada fica bem feito. Faça por prioridade: HTTPS, backup, monitoramento. Aí dorme tranquilo.

A Apêndice — Comandos de referência rápida

Sistema

```
uname -a           # kernel + arquitetura
lsb_release -a     # distro
free -h           # memória
df -h             # disco
top | htop        # processos em tempo real
journalctl -u docker -f # logs do daemon Docker
```

Rede e firewall

```
ip addr show      # IPs das interfaces
ss -tlnp         # portas TCP escutando
sudo iptables -L INPUT -n --line-numbers
sudo iptables -I INPUT N -p tcp --dport PORTA -j ACCEPT
sudo netfilter-persistent save
curl -v http://IP:PORTA/health
```

Swap

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
swapon --show      # conferir
```

Docker

```
docker ps                # containers rodando
docker ps -a             # inclui parados
docker images            # imagens locais
docker compose up -d     # sobe em background
docker compose down      # derruba (mantém volumes)
docker compose down -v   # derruba E apaga volumes!
docker compose logs -f SERV # tail de logs
docker exec -it CONTAINER sh # shell dentro do container
docker stats             # CPU/mem em tempo real
docker system prune -af  # limpa lixo
```

Postgres dentro do container

```
docker exec -it qnf-postgres psql -U qnf_user -d qnf_db
# Comandos psql úteis:
\l                # lista databases
\dt               # lista tabelas
\d nome           # descreve tabela
\du               # lista usuários
\q               # sai
```

Prisma

```
npx prisma generate      # gera o client
npx prisma migrate dev   # cria migration (DEV)
npx prisma migrate deploy # aplica migrations (PROD)
npx prisma studio        # GUI web pra explorar o banco
```

Expo / Metro

```
npm install              # instala deps
npm run start:dev-client:tunnel # Metro com tunnel
npx expo start --dev-client   # Metro local (mesma rede)
eas build --profile development --platform ios # gera dev client iOS
```


Termo	Significado
VCN	Virtual Cloud Network — rede virtual da Oracle Cloud, equivalente ao VPC da AWS.
Security List	Conjunto de regras de firewall no nível do VCN. Aplica-se ao tráfego antes mesmo de chegar na VM.
Ingress / Egress	Tráfego entrando (ingress) ou saindo (egress) da rede.
CIDR	Classless Inter-Domain Routing — notação <code>IP/máscara</code> . <code>0.0.0.0/0</code> = qualquer IP. <code>192.168.1.0/24</code> = a rede 192.168.1.x.
iptables	Frontend tradicional para o netfilter (firewall) do kernel Linux. Substituído gradualmente por nftables.
chain INPUT	Conjunto de regras avaliadas para pacotes destinados ao próprio host. Existem também FORWARD (rotear) e OUTPUT (sair).
OOM killer	Out Of Memory killer — mecanismo do kernel que mata processos quando a RAM acaba.
swap	Memória virtual em disco. Estende a RAM com altíssimo custo de latência.
swappiness	Parâmetro kernel (0–100) que controla quão "agressivamente" o kernel move páginas pra swap. Default 60.
healthcheck	Comando que o Docker executa periodicamente para saber se um container está "saudável". Outros serviços podem esperar isso (depends_on).
multi-stage build	Padrão Dockerfile com vários <code>FROM</code> ; estágio final copia só o necessário dos anteriores, resultando em imagem menor.
BullMQ	Biblioteca Node para filas de jobs em background, usando Redis como backend. Sucessora do Bull.
Prisma	ORM TypeScript-first com schema declarativo, migrations versionadas e client tipado gerado automaticamente.
Fastify	Framework HTTP Node.js focado em performance e DX. Alternativa moderna ao Express.
JWT	JSON Web Token — formato de token autenticável (assinado) usado em APIs stateless.

Access / Refresh token	Padrão de autenticação: access token de curta duração para chamadas; refresh de longa duração para renovar o access sem login.
Pluggy	Provider open finance brasileiro. API que conecta contas bancárias e devolve transações categorizadas.
EAS Build	Expo Application Services — build de apps nativos na nuvem da Expo. Substitui ter Xcode/Android Studio local.
Expo Dev Client	Versão custom do Expo Go que inclui suas dependências nativas. Necessário quando você usa libs além do Expo SDK base.
Heredoc	Construção shell <code><<EOF ... EOF</code> que trata o conteúdo entre os delimitadores como input do comando.
tini	Init system minimalista. Roda como PID 1 em containers pra lidar com sinais e processos zumbis.
Alpine	Distribuição Linux super-leve baseada em musl libc e BusyBox. Imagem Docker de ~5 MB.
UFW	Uncomplicated Firewall — wrapper amigável em cima de iptables. Comum no Ubuntu desktop.
OCI	Oracle Cloud Infrastructure — o nome formal da cloud da Oracle.

— Fim da aula —

Quase Nada Software House · 2026